Advanced Data Computing (405 @ LU) Notes

Abhishek Chakraborty

2026-07-24

Table of contents

Welcome

This is a Quarto book.

To learn more about Quarto books visit https://quarto.org/docs/books.

Part I

SQL

1 Accessing Databases and Database Tables

1.1 resources

- R for Data Science (2e) - Chapter 21
- Modern Data Science with R (3e) - Chapters 15 and 16
- Notes from Database Systems at UC Berkeley
- Introduction to dbplyr
- Hands-on Database (2e) - Steve Conger
- CMSC 225 - Database Techniques and Modeling (F'26 and S'27)

1.2 load packages

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

1.3 databases and accessing them

```
# access airlines database

con_air <- mdsr::dbConnect_scidb("airlines")
```

```r
# access wai database

con_wai <- DBI::dbConnect(
  RMariaDB::MariaDB(),
  dbname = "wai",
  host = "scidb.smith.edu",
  user = "waiuser",
  password = "smith_waiDB")
```

1.4 database tables and accessing them

```r
# lists tables in the database

DBI::dbListTables(con_air)
```

```
--- lists tables in the database

SHOW TABLES;
```

```r
# access flights table from the airlines database

flights <- dplyr::tbl(con_air, "flights")
```

```
DESCRIBE flights;
```

```r
# access carriers table from the airlines database

carriers <- dplyr::tbl(con_air, "carriers")
```

```
DESCRIBE carriers;
```

```r
# carriers lives in the SQL database and is linked remotely

carriers |>
  utils::object.size() |>
  print(units = "Kb")

# nycflights13::flights |>
```

```
#   object.size() |>
#   print(units = "Mb")
```

```
# carriers lives in R

carriers |>
  dplyr::collect() |>
  utils::object.size() |>
  print(units = "Kb")
```

```
carries_df <- carriers |>  dplyr::collect()
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

2 Writing SQL queries

2.1 load packages and connect to database server

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

```
# access airlines database

con_air <- mdsr::dbConnect_scidb("airlines")

DBI::dbListTables(con_air)   # list tables in database
```

```
# access flights table from the airlines database

flights <- dplyr::tbl(con_air, "flights")
```

```
DESCRIBE flights;
```

2.2 SQL in R: Part 1 - the 'dbplyr' R package will directly translate 'dplyr' code into SQL

```
yrs <- flights |>
  summarize(min_year = min(year, na.rm = TRUE),
            max_year = max(year, na.rm = TRUE))

yrs
```

```
dplyr::show_query(yrs)
```

```
la_bos <- flights |>
  filter(year == 2013 &
           ((origin == "LAX" & dest == "BOS") |
           (origin == "BOS" & dest == "LAX")))

la_bos
```

```
dplyr::show_query(la_bos)
```

```
la_bos <- la_bos |>
  collect()

la_bos
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

2.3 SQL in R: Part 2 - using the DBI R package to write SQL queries in r code chunks

```
DBI::dbGetQuery(con_air,
                "SELECT * FROM flights LIMIT 8;")
```

```
DBI::dbGetQuery(con_air,
  "SELECT year, count(*) AS num_flights FROM flights GROUP BY year ORDER BY num_flights;")
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

2.4 SQL in R: Part 3 - writing SQL queries in sql code chunks

```sql
SELECT MIN(year) AS min_year,
       MAX(year) AS max_year
FROM flights;
```

```sql
SELECT *
FROM flights
WHERE (year = 2013 AND
      ((origin = 'LAX' AND dest = 'BOS') OR
      (origin = 'BOS' AND dest = 'LAX')));
```

```sql
SELECT *
FROM flights
LIMIT 2;
```

```sql
SELECT year,
       COUNT(*) AS num_flights
FROM flights
GROUP BY year
ORDER BY num_flights;
```

Queries in SQL start with the **SELECT** keyword and consist of several clauses, which have to be written in this order:

- **SELECT** allows you to list the columns, or functions operating on columns, that you want to retrieve.
- **FROM** specifies the table where the data are.
- **JOIN** allows you to stitch together two or more tables using a key.
- **WHERE** allows you to extract rows according to some criteria.
- **GROUP BY** allows you to aggregate the records according to some shared value.
- **HAVING** is like a WHERE clause that operates on the result set—not the original rows themselves.
- **ORDER BY** orders the rows of the result set.
- **LIMIT** restricts the number of rows in the output.

```sql
DESCRIBE flights;
```

```
-- display the first ten rows of the flights table
```

```
-- display year, month, day, origin, dep_time, sched_dep_time, dep_delay for the first ten r
```

```
-- count the total number of rows in the flights table
```

```
-- extract all info about flights that flew from Appleton (ATW) to Chicago (ORD) in 2013
```

```
-- count the number of flights from Appleton (ATW) for each year
```

```
-- which carriers flew from Appleton (ATW) to Chicago (ORD)
-- how many flights did they do per year
```

```
-- for flights that flew from Appleton (ATW) to Chicago (ORD)
-- what was the latest flight for each carrier
```

```
-- for flights that flew from Appleton (ATW)
-- which destination had the lowest average arrival delay time
```

```
-- for flights that flew from Appleton (ATW)
-- consider only destinations with at least 2000 flights from Appleton (ATW)
-- which destination had the lowest average arrival delay time
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

3 Joins, Unions, and Subqueries

3.1 load packages and connect to database server

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

```
# access airlines database

con_air <- mdsr::dbConnect_scidb("airlines")

DBI::dbListTables(con_air)   # list tables in database
```

```
# access tables from the airlines database

flights <- dplyr::tbl(con_air, "flights")

carriers <- dplyr::tbl(con_air, "carriers")

airports <- dplyr::tbl(con_air, "airports")
```

```
DESCRIBE flights;
```

```
SELECT * FROM flights LIMIT 10;
```

```
DESCRIBE carriers;
```

```
SELECT * FROM carriers LIMIT 100;
```

```
DESCRIBE airports;
```

```
SELECT * FROM airports LIMIT 100;
```

3.2 Joins

In general, there are four pieces of information that you need to specify in order to join two tables:

- name of the first table
- type of join
- name of the second table
- condition(s) of the join

The following join types are available in MySQL dialect of SQL:

- **JOIN**
- **LEFT JOIN**
- **RIGHT JOIN**
- **CROSS JOIN**

```
-- access airport names for flights that flew out of Appleton in 2013
```

```
-- access both airport and carrier names for flights that flew out of Appleton in 2013
```

```
-- using LEFT JOIN
```

3.3 Unions

```
-- extract all flights that flew from ATW to MSP and from JFK to ORD on April 14, 2013
```

3.4 Subqueries

```
-- count the number of flights that satisfy the criteria above
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

4 Creating Databases

4.1 load packages and connect to database server

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
library(duckdb)
```

As an example, we will consider the Saturday Night Live datasets available on the **snldb** GitHub repo. Data is scraped from SNL archives and http://www.imdb.com/title/tt0072562 by Hendrik Hilleckes and Colin Morris.

```
con_snl <- DBI::dbConnect(duckdb::duckdb(), dbdir = "snl_db")
```

```
SHOW TABLES;
```

4.2 accessing data (csv files) from GitHub (or, elsewhere)

Notice that there are eleven **.csv** files available in the **output** folder. Let's look at the **casts** dataset in R, so that we understand the data we want to load into the database.

```
# read in the data

casts <- readr::read_csv("")
```

```
glimpse(casts)
```

```
summary(casts)
```

```
# save the dataset as a .csv file on your (local) computer

write_csv(casts, "casts.csv")
```

4.3 data types in SQL

Consult data types for details.

4.4 importing data

Once the database is set up, you will be ready to import .csv files into the database as tables. Importing .csv files as tables requires a series of steps:

- a **USE** statement that ensures we are in the right database.
- a series of **DROP TABLE** statements that drop any old tables with the same names as the ones we are going to create.
- a series of **CREATE TABLE** statements that specify the table structures and field types.
- a series of **COPY** (or **LOAD DATA** for MySQL) statements that read the data from the .csv files into the appropriate tables.

```
USE snl_db;
```

```
DROP TABLE IF EXISTS casts;
```

```
CREATE TABLE casts (
  aid VARCHAR(255) NOT NULL DEFAULT ' ',
  sid SMALLINT NOT NULL DEFAULT 0,
  featured BOOLEAN NOT NULL DEFAULT FALSE,  /* use TINYINT(1) in MySQL instead of BOOLEAN) *,
  first_epid INTEGER DEFAULT NULL,
  last_epid INTEGER DEFAULT NULL,
  update_anchor BOOLEAN NOT NULL DEFAULT FALSE,
  n_episodes SMALLINT NOT NULL DEFAULT 0,
  season_fraction DOUBLE NOT NULL DEFAULT 0,
  PRIMARY KEY (sid, aid)
);
```

```
COPY casts FROM 'casts.csv' (FORMAT CSV, HEADER, NULL 'NA');
```

4.5 checks

```
SHOW DATABASES;
```

```
SHOW TABLES;
```

```
DESCRIBE casts;
```

```
SELECT * FROM casts LIMIT 10;
```

4.6 shortcut (not accepted for Project 2)

```
# read in the data

episodes <- readr::read_csv("")

glimpse(episodes)
```

```
# save the dataset as a .csv file on your (local) computer

write_csv(episodes, "episodes.csv")
```

```
duckdb_read_csv(con = con_snl, name = "episodes", files = "episodes.csv")
```

```
SHOW TABLES;
```

```
DESCRIBE episodes;
```

```
SELECT * FROM episodes LIMIT 10;
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_snl, shutdown = TRUE)
```

Part II

Text Analysis

5 Case Study of Shakepearse's Macbeth using Regular Expressions

5.1 resources

- Text Mining with R
- Modern Data Science with R (3e) - Chapter 19

5.2 packages

```
library(tidyverse)
```

5.3 data

Our data comes from Project Gutenberg, a large repository of free eBooks. We will be working with the text of Shakespeare's play Macbeth, which is available at https://www.gutenberg.org/cache/epub/1129/pg1129.txt. The text is in the public domain, so we are free to use it for our analysis.

```
library(gutenbergr)

data("gutenberg_metadata")

macbeth_all <- gutenberg_metadata |>
  filter(str_detect(title, "Macbeth"))

book <- gutenberg_download(1129, meta_fields = "title")
```

For our case study, we will be working with the raw text of the play, which is available in the `mdsr` package.

```
library(mdsr)

data(Macbeth_raw)

?Macbeth_raw
```

```
macbeth <- Macbeth_raw |>
  str_split("\r\n") |>
  pluck(1)

length(macbeth)

macbeth[300:310]
```

5.4 strings and regular expressions

```
# how many times does the character Macbeth speak in the play?

macbeth_lines <- macbeth |>
  str_subset("  MACBETH")

length(macbeth_lines)

head(macbeth_lines)
```

```
macbeth |>
  str_which("  MACBETH")
```

5.5 about regular expressions grammar

5.5.1 metacharacter

```
macbeth |>
  str_subset("MAC.") |>
  head()
```

```
macbeth |>
  str_subset("MACBETH\\.") |>
  head()
```

5.5.2 character sets

```
macbeth |>
  str_subset("MAC[B-Z]") |>
  head()
```

5.5.3 alternation

```
macbeth |>
  str_subset("MAC(B|D)") |>
  head()
```

5.5.4 anchors (power-dollar)

```
macbeth |>
  str_subset("^  MAC[B-Z]") |>
  head()
```

```
macbeth |>
  str_subset("\\.$") |>
  head()
```

5.5.5 repetitions

```
# ?

macbeth |>
  str_subset("^ ?MAC[B-Z]") |>
  head()
```

```
# *

macbeth |>
  str_subset("^ *MAC[B-Z]") |>
  head()
```

```
# +

macbeth |>
  str_subset("^ +MAC[B-Z]") |>
  head()
```

5.6 a case study - life and death in Macbeth and speaking patterns

```
macbeth_chars <- tribble(
  ~name, ~regexp,
  "Macbeth", "  MACBETH\\.",
  "Lady Macbeth", "  LADY MACBETH\\.",
  "Banquo", "  BANQUO\\.",
  "Duncan", "  DUNCAN\\.",
) |>
  mutate(speaks = purrr::map(regexp, str_detect, string = macbeth))
```

```
speaker_freq <- macbeth_chars |>
  unnest(cols = speaks) |>
  mutate(
    line = rep(1:length(macbeth), 4),
    speaks = as.numeric(speaks)
  ) |>
  filter(line > 218 & line < 3172)

glimpse(speaker_freq)
```

```
acts <- tibble(
  line = str_which(macbeth, "^ACT [I|V]+"),
  line_text = str_subset(macbeth, "^ACT [I|V]+"),
  labels = str_extract(line_text, "^ACT [I|V]+")
)
```

```r
ggplot(data = speaker_freq, aes(x = line, y = speaks)) +
  geom_smooth(
    aes(color = name), method = "loess",
    se = FALSE, span = 0.4
  ) +
  geom_vline(
    data = acts,
    aes(xintercept = line),
    color = "darkgray", lty = 3
  ) +
  geom_text(
    data = acts,
    aes(y = 0.085, label = labels),
    hjust = "left", color = "darkgray"
  ) +
  ylim(c(0, NA)) +
  xlab("Line Number") +
  ylab("Proportion of Speeches") +
  scale_color_colorblind()
```

6 Case Study of Donald Trump's tweets during the 2016 US presidential election

6.1 resources

- Trump Twitter Archive
- Todd Vaziri's Tweet
- David Robinson's Blog Post
- Text Mining With R

6.2 packages

```
library(tidyverse)
library(scales)
library(tidytext)
library(textdata)
library(dslabs)
```

6.3 data

Our data come from the *dslabs* package.

```
data("trump_tweets")

?trump_tweets

glimpse(trump_tweets)
```

```
range(trump_tweets$created_at)
```

```
trump_tweets$text[16413]
```

```
trump_tweets |>
  count(source) |>
  arrange(desc(n))
```

For our case study, we are interested in what happened during the 2016 campaign, so for this analysis we will focus on what was tweeted between the day Trump announced his campaign and election day. Also, we only keep tweets from Android or iPhone, and filter out retweets.

```
campaign_tweets <- trump_tweets |>
  filter(source %in% paste("Twitter for", c("Android", "iPhone")) &
           created_at >= ymd("2015-06-17") &
           created_at < ymd("2016-11-08")) |>
  mutate(source = str_remove(source, pattern = "Twitter for ")) |>
  filter(!is_retweet) |>
  arrange(created_at) |>
  as_tibble()
```

```
campaign_tweets |>
  count(source)
```

6.4 some initial exploration

We explore the possibility that two different groups were tweeting from these devices.

```
campaign_tweets |>
  mutate(hour = hour(with_tz(created_at, "EST"))) |>
  count(source, hour)
```

```
android_vs_iphone_hourly_props <- campaign_tweets |>
  mutate(hour = hour(with_tz(created_at, "EST"))) |>
  count(source, hour) |>
  group_by(source) |>
  mutate(percent = n / sum(n))

android_vs_iphone_hourly_props
```

```
android_vs_iphone_hourly_props |>
  ggplot(aes(x = hour, y = percent, color = source)) +
  geom_line() +
  geom_point() +
  scale_y_continuous(labels = percent_format()) +
  labs(x = "Hour of day (EST)", y = "% of tweets", color = "")
```

6.5 tokens

```
i <- 3008

campaign_tweets$text[i]
```

```
campaign_tweets[i,] |>
  tidytext::unnest_tokens(output = word, input = text, token = "words") |>
  dplyr::pull(word)
```

```
links_to_pics <- "(https://t.co/[A-Za-z\\d]+)|(http://t.co/[A-Za-z\\d]+)"

campaign_tweets[i,] |>
  mutate(text = str_remove_all(text, pattern = links_to_pics)) |>
  tidytext::unnest_tokens(output = word, input = text, token = "words") |>
  dplyr::pull(word)
```

```
tweet_words <- campaign_tweets |>
  mutate(text = str_remove_all(text, pattern = links_to_pics))  |>
  tidytext::unnest_tokens(output = word, input = text, token = "words")
```

```
tweet_words |>
  count(word) |>
  arrange(desc(n))
```

6.6 stop words

```r
data(stop_words)

?stop_words
```

```r
tweet_words <- campaign_tweets |>
  mutate(text = str_remove_all(text, pattern = links_to_pics))  |>
  tidytext::unnest_tokens(output = word, input = text, token = "words") |>
  filter(!(word %in% stop_words$word))

tweet_words |>
  count(word)
```

```r
tweet_words <- campaign_tweets |>
  mutate(text = str_remove_all(text, links_to_pics))  |>
  tidytext::unnest_tokens(output = word, input = text, token = "words") |>
  filter(!(word %in% stop_words$word)) |>
  filter(!(str_detect(word, pattern = "^\\d+")))

tweet_words |>
  count(word)

# |>
#   arrange(desc(n))
```

6.7 further exploration

```r
android_vs_iphone <- tweet_words |>
  count(word, source)

android_vs_iphone
```

```r
android_vs_iphone <- tweet_words |>
  count(word, source) |>
  pivot_wider(names_from = "source", values_from = "n", values_fill = 0)

android_vs_iphone
```

```r
android_vs_iphone <- tweet_words |>
  count(word, source) |>
  pivot_wider(names_from = "source", values_from = "n", values_fill = 0) |>
  mutate(p_a = (Android + 0.5)/(sum(Android) + 0.5),
         p_i = (iPhone + 0.5)/(sum(iPhone) + 0.5),
         ratio = p_a / p_i)

android_vs_iphone
```

```r
# for words appearing at least 100 times in total, here are the ten highest percent differenc

top_10_android <- android_vs_iphone |>
  filter(Android + iPhone >= 100) |>
  arrange(desc(ratio)) |>
  slice_head(n = 10)

top_10_android
```

```r
# and the top ten for iPhone

top_10_iphone <- android_vs_iphone |>
  filter(Android + iPhone >= 100) |>
  arrange(ratio)|>
  slice_head(n = 10)

top_10_iphone
```

```r
top_10_android |>
  rbind(top_10_iphone) |>
  dplyr::select(word, iPhone, Android, ratio) |>
  mutate(log_ratio = log(ratio)) |>
  mutate(device = rep(c("Android", "iPhone"), each = 10)) |>
  ggplot() +
  geom_col(aes(x = log_ratio, y = fct_reorder(word, log_ratio), fill = device))
```

6.8 sentiment analysis

```r
tidytext::get_sentiments("bing")
```

```
tidytext::get_sentiments("afinn")

tidytext::get_sentiments("nrc")
```

```
nrc <- tidytext::get_sentiments("nrc")

nrc |> count(sentiment)
```

```
tweet_words |>
  left_join(nrc, by = "word") |>
  dplyr::select(source, word, sentiment) |>
  sample_n(5)
```

```
sentiment_counts <- tweet_words |>
  left_join(nrc, by = "word", relationship = "many-to-many") |>
  count(source, sentiment)

sentiment_counts
```

```
sentiment_counts <- tweet_words |>
  left_join(nrc, by = "word", relationship = "many-to-many") |>
  count(source, sentiment) |>
  pivot_wider(names_from = "source", values_from = "n") |>
  mutate(sentiment = replace_na(sentiment, replace = "none"))

sentiment_counts
```

```
sentiment_counts <- sentiment_counts |>
  mutate(p_a = Android/sum(Android),
         p_i = iPhone/sum(iPhone),
         ratio = p_a/p_i) |>
  arrange(desc(ratio))

sentiment_counts
```

```
ordered_levels <- sentiment_counts$sentiment

android_vs_iphone |>
  filter(Android + iPhone >= 10) |>
  left_join(nrc, by = "word") |>
```

```
  mutate(sentiment = factor(sentiment, levels = sentiment_counts$sentiment)) |>
  group_by(sentiment) |>
  slice_max(abs(log(ratio)), n = 10) |>
  ungroup() |>
  mutate(word = reorder(word, -ratio)) |>
  ggplot(aes(word, ratio, fill = ratio < 1)) +
  geom_col(show.legend = FALSE) +
  scale_y_log10() +
  facet_wrap(~sentiment, scales = "free_x", nrow = 2) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
```

7 Case Study of Data Science Papers from arXiv.org

7.1 resources

- arXiv.org

7.2 packages

```
library(tidyverse)
library(scales)
library(tidytext)
library(textdata)
library(mdsr)
```

7.3 data

We can import data using the **aRxiv** package.

```
library(aRxiv)

df <- arxiv_search(
  query = 'ti:"Data Science"',
  limit = 20000,
  batchsize = 100
)

glimpse(df)
```

For this case study we will use some already-collected data from the **mdsr** package.

```
library(mdsr)

data(DataSciencePapers)

?DataSciencePapers

glimpse(DataSciencePapers)
```

7.4 initial data exploration and cleaning

```
DataSciencePapers <- DataSciencePapers |>
  mutate(
    submitted = lubridate::ymd_hms(submitted),
    updated = lubridate::ymd_hms(updated)
  )

glimpse(DataSciencePapers)
```

```
mosaic::tally(~ year(submitted), data = DataSciencePapers)
```

```
DataSciencePapers |>
  filter(id == "1809.02408v2") |>
  glimpse()
```

```
DataSciencePapers |>
  group_by(primary_category) |>
  count()
```

```
DataSciencePapers <- DataSciencePapers |>
  mutate(field = str_extract(primary_category, pattern = "^[a-z,-]+"))

mosaic::tally(x = ~field, margins = TRUE, data = DataSciencePapers) |>
  sort()
```

7.5 corpora and tokenization

```
DataSciencePapers |>
  unnest_tokens(output = word, input = abstract, token = "words") |>
  count(id, word, sort = TRUE)
```

```
arxiv_words <- DataSciencePapers |>
  unnest_tokens(output = word, input = abstract, token = "words") |>
  anti_join(tidytext::get_stopwords(), by = "word")

arxiv_words |>
  count(id, word, sort = TRUE)
```

```
arxiv_abstracts <- arxiv_words |>
  group_by(id) |>
  summarize(abstract_clean = paste(word, collapse = " "))

arxiv_papers <- DataSciencePapers |>
  left_join(arxiv_abstracts, by = "id")
```

7.6 wordcloud

```
library(wordcloud)

set.seed(4302026)
arxiv_papers |>
  pull(abstract_clean) |>
  wordcloud(
    max.words = 40,
    scale = c(8, 1),
    colors = topo.colors(n = 30),
    random.color = TRUE
  )
```

7.7 bigrams

```
arxiv_bigrams <- arxiv_papers |>
  unnest_tokens(
```

```
    output = arxiv_bigram,
    input = abstract_clean,
    token = "ngrams",
    n = 2
  ) |>
  dplyr::select(arxiv_bigram, id)

arxiv_bigrams

arxiv_bigrams |>
  count(arxiv_bigram, sort = TRUE)
```

7.8 tf-idf and document term matrices

```
arxiv_words |>
  count(word) |>
  arrange(desc(n)) |>
  head()
```

```
tidy_DTM <- arxiv_words |>
  count(id, word) |>
  tidytext::bind_tf_idf(word, id, n)

tidy_DTM
```

```
tidy_DTM |>
  arrange(desc(tf)) |>
  head()
```

```
tidy_DTM |>
  arrange(desc(idf), desc(n)) |>
  head()
```

```
tidy_DTM |>
  filter(word == "wildfire")
```

```
arxiv_papers |>
  pull(abstract) |>
```

```
  str_subset("wildfire") |>
  strwrap() |>
  head()
```

```
tidy_DTM |>
  filter(word == "implications")
```

```
tidy_DTM |>
  filter(id == "1809.02408v2") |>
  arrange(desc(tf_idf)) |>
  head()
```

```
tm_DTM <- arxiv_words |>
  count(id, word) |>
  tidytext::cast_dtm(document = id, term = word, value = n)

tm_DTM
```

Part III

Geospatial Data

8 Accessing Databases and Database Tables

8.1 resources

- R for Data Science (2e) - Chapter 21
- Modern Data Science with R (3e) - Chapters 15 and 16
- Notes from Database Systems at UC Berkeley
- Introduction to dbplyr
- Hands-on Database (2e) - Steve Conger
- CMSC 225 - Database Techniques and Modeling (F'26 and S'27)

8.2 load packages

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

8.3 databases and accessing them

```
# access airlines database

con_air <- mdsr::dbConnect_scidb("airlines")
```

```r
# access wai database

con_wai <- DBI::dbConnect(
  RMariaDB::MariaDB(),
  dbname = "wai",
  host = "scidb.smith.edu",
  user = "waiuser",
  password = "smith_waiDB")
```

8.4 database tables and accessing them

```r
# lists tables in the database

DBI::dbListTables(con_air)
```

```
--- lists tables in the database

SHOW TABLES;
```

```r
# access flights table from the airlines database

flights <- dplyr::tbl(con_air, "flights")
```

```
DESCRIBE flights;
```

```r
# access carriers table from the airlines database

carriers <- dplyr::tbl(con_air, "carriers")
```

```
DESCRIBE carriers;
```

```r
# carriers lives in the SQL database and is linked remotely

carriers |>
  utils::object.size() |>
  print(units = "Kb")

# nycflights13::flights |>
```

```
#   object.size() |>
#   print(units = "Mb")
```

```
# carriers lives in R

carriers |>
  dplyr::collect() |>
  utils::object.size() |>
  print(units = "Kb")
```

```
carries_df <- carriers |>  dplyr::collect()
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

9 Writing SQL queries

9.1 load packages and connect to database server

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

```
# access airlines database

con_air <- mdsr::dbConnect_scidb("airlines")

DBI::dbListTables(con_air)   # list tables in database
```

```
# access flights table from the airlines database

flights <- dplyr::tbl(con_air, "flights")
```

```
DESCRIBE flights;
```

9.2 SQL in R: Part 1 - the 'dbplyr' R package will directly translate 'dplyr' code into SQL

```
yrs <- flights |>
  summarize(min_year = min(year, na.rm = TRUE),
            max_year = max(year, na.rm = TRUE))

yrs
```

```
dplyr::show_query(yrs)
```

```
la_bos <- flights |>
  filter(year == 2013 &
           ((origin == "LAX" & dest == "BOS") |
           (origin == "BOS" & dest == "LAX")))

la_bos
```

```
dplyr::show_query(la_bos)
```

```
la_bos <- la_bos |>
  collect()

la_bos
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

9.3 SQL in R: Part 2 - using the DBI R package to write SQL queries in r code chunks

```
DBI::dbGetQuery(con_air,
                "SELECT * FROM flights LIMIT 8;")
```

```
DBI::dbGetQuery(con_air,
  "SELECT year, count(*) AS num_flights FROM flights GROUP BY year ORDER BY num_flights;")
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

9.4 SQL in R: Part 3 - writing SQL queries in sql code chunks

```sql
SELECT MIN(year) AS min_year,
       MAX(year) AS max_year
FROM flights;
```

```sql
SELECT *
FROM flights
WHERE (year = 2013 AND
      ((origin = 'LAX' AND dest = 'BOS') OR
      (origin = 'BOS' AND dest = 'LAX')));
```

```sql
SELECT *
FROM flights
LIMIT 2;
```

```sql
SELECT year,
       COUNT(*) AS num_flights
FROM flights
GROUP BY year
ORDER BY num_flights;
```

Queries in SQL start with the **SELECT** keyword and consist of several clauses, which have to be written in this order:

- **SELECT** allows you to list the columns, or functions operating on columns, that you want to retrieve.
- **FROM** specifies the table where the data are.
- **JOIN** allows you to stitch together two or more tables using a key.
- **WHERE** allows you to extract rows according to some criteria.
- **GROUP BY** allows you to aggregate the records according to some shared value.
- **HAVING** is like a WHERE clause that operates on the result set—not the original rows themselves.
- **ORDER BY** orders the rows of the result set.
- **LIMIT** restricts the number of rows in the output.

```sql
DESCRIBE flights;
```

```
-- display the first ten rows of the flights table
```

```
-- display year, month, day, origin, dep_time, sched_dep_time, dep_delay for the first ten r
```

```
-- count the total number of rows in the flights table
```

```
-- extract all info about flights that flew from Appleton (ATW) to Chicago (ORD) in 2013
```

```
-- count the number of flights from Appleton (ATW) for each year
```

```
-- which carriers flew from Appleton (ATW) to Chicago (ORD)
-- how many flights did they do per year
```

```
-- for flights that flew from Appleton (ATW) to Chicago (ORD)
-- what was the latest flight for each carrier
```

```
-- for flights that flew from Appleton (ATW)
-- which destination had the lowest average arrival delay time
```

```
-- for flights that flew from Appleton (ATW)
-- consider only destinations with at least 2000 flights from Appleton (ATW)
-- which destination had the lowest average arrival delay time
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

Part IV

Code Performance

10 Accessing Databases and Database Tables

10.1 resources

- R for Data Science (2e) - Chapter 21
- Modern Data Science with R (3e) - Chapters 15 and 16
- Notes from Database Systems at UC Berkeley
- Introduction to dbplyr
- Hands-on Database (2e) - Steve Conger
- CMSC 225 - Database Techniques and Modeling (F'26 and S'27)

10.2 load packages

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

10.3 databases and accessing them

```
# access airlines database

con_air <- mdsr::dbConnect_scidb("airlines")
```

```
# access wai database

con_wai <- DBI::dbConnect(
  RMariaDB::MariaDB(),
  dbname = "wai",
  host = "scidb.smith.edu",
  user = "waiuser",
  password = "smith_waiDB")
```

10.4 database tables and accessing them

```
# lists tables in the database

DBI::dbListTables(con_air)
```

```
--- lists tables in the database

SHOW TABLES;
```

```
# access flights table from the airlines database

flights <- dplyr::tbl(con_air, "flights")
```

```
DESCRIBE flights;
```

```
# access carriers table from the airlines database

carriers <- dplyr::tbl(con_air, "carriers")
```

```
DESCRIBE carriers;
```

```
# carriers lives in the SQL database and is linked remotely

carriers |>
  utils::object.size() |>
  print(units = "Kb")

# nycflights13::flights |>
```

```
#   object.size() |>
#   print(units = "Mb")
```

```
# carriers lives in R

carriers |>
  dplyr::collect() |>
  utils::object.size() |>
  print(units = "Kb")
```

```
carries_df <- carriers |>  dplyr::collect()
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

11 Writing SQL queries

11.1 load packages and connect to database server

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

```
# access airlines database

con_air <- mdsr::dbConnect_scidb("airlines")

DBI::dbListTables(con_air)   # list tables in database
```

```
# access flights table from the airlines database

flights <- dplyr::tbl(con_air, "flights")
```

```
DESCRIBE flights;
```

11.2 SQL in R: Part 1 - the 'dbplyr' R package will directly translate 'dplyr' code into SQL

```
yrs <- flights |>
  summarize(min_year = min(year, na.rm = TRUE),
            max_year = max(year, na.rm = TRUE))

yrs
```

```
dplyr::show_query(yrs)
```

```
la_bos <- flights |>
  filter(year == 2013 &
           ((origin == "LAX" & dest == "BOS") |
           (origin == "BOS" & dest == "LAX")))

la_bos
```

```
dplyr::show_query(la_bos)
```

```
la_bos <- la_bos |>
  collect()

la_bos
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

11.3 SQL in R: Part 2 - using the DBI R package to write SQL queries in r code chunks

```
DBI::dbGetQuery(con_air,
                "SELECT * FROM flights LIMIT 8;")
```

```
DBI::dbGetQuery(con_air,
  "SELECT year, count(*) AS num_flights FROM flights GROUP BY year ORDER BY num_flights;")
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

11.4 SQL in R: Part 3 - writing SQL queries in sql code chunks

```sql
SELECT MIN(year) AS min_year,
       MAX(year) AS max_year
FROM flights;
```

```sql
SELECT *
FROM flights
WHERE (year = 2013 AND
      ((origin = 'LAX' AND dest = 'BOS') OR
      (origin = 'BOS' AND dest = 'LAX')));
```

```sql
SELECT *
FROM flights
LIMIT 2;
```

```sql
SELECT year,
       COUNT(*) AS num_flights
FROM flights
GROUP BY year
ORDER BY num_flights;
```

Queries in SQL start with the **SELECT** keyword and consist of several clauses, which have to be written in this order:

- **SELECT** allows you to list the columns, or functions operating on columns, that you want to retrieve.
- **FROM** specifies the table where the data are.
- **JOIN** allows you to stitch together two or more tables using a key.
- **WHERE** allows you to extract rows according to some criteria.
- **GROUP BY** allows you to aggregate the records according to some shared value.
- **HAVING** is like a WHERE clause that operates on the result set—not the original rows themselves.
- **ORDER BY** orders the rows of the result set.
- **LIMIT** restricts the number of rows in the output.

```sql
DESCRIBE flights;
```

```
-- display the first ten rows of the flights table
```

```
-- display year, month, day, origin, dep_time, sched_dep_time, dep_delay for the first ten r
```

```
-- count the total number of rows in the flights table
```

```
-- extract all info about flights that flew from Appleton (ATW) to Chicago (ORD) in 2013
```

```
-- count the number of flights from Appleton (ATW) for each year
```

```
-- which carriers flew from Appleton (ATW) to Chicago (ORD)
-- how many flights did they do per year
```

```
-- for flights that flew from Appleton (ATW) to Chicago (ORD)
-- what was the latest flight for each carrier
```

```
-- for flights that flew from Appleton (ATW)
-- which destination had the lowest average arrival delay time
```

```
-- for flights that flew from Appleton (ATW)
-- consider only destinations with at least 2000 flights from Appleton (ATW)
-- which destination had the lowest average arrival delay time
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```